

**Robot Sumo Simulation:
Requirements Document**

Team Members

Adebare Kofo-Abayomi: akofoaabyomi2022@my.fit.edu

Avlynn Wade: awade2024@my.fit.edu

Faysal Zaki: fzaki2023@my.fit.edu

Joshua Oziel Fernández Alvarado: jfernandezal2023@my.fit.edu

Client & Faculty Advisor

Department of Electrical Engineering and Computer Science

Dr. Hanks, thanks@fit.edu

Table of Contents

1. Introduction
 - 1.1 Purpose
 - 1.2 Scope
 - 1.3 Definitions, acronyms, and abbreviations
 - 1.4 References
 - 1.5 Overview
2. Overall description
 - 2.1 Product perspective
 - 2.1.1 System interfaces
 - 2.1.2 User interfaces
 - 2.1.3 Hardware interfaces
 - 2.1.4 Software interfaces
 - 2.1.5 Communication interfaces
 - 2.1.6 Memory
 - 2.1.7 Operations
 - 2.2 Product functions
 - 2.3 User characteristics
 - 2.4 Constraints
 - 2.5 Assumptions and dependencies
3. Specific Requirements
 - 3.1 External interface requirements
 - 3.1.1 User interfaces
 - 3.1.2 Hardware interfaces
 - 3.1.3 Software interfaces
 - 3.1.4 Communications interfaces
 - 3.2 Functional requirements
 - 3.2.1 Match Configuration
 - 3.2.2 Single-Agent RL Training
 - 3.2.3 Progressive Training
 - 3.2.4 Pre-Trained Agent Visualization
 - 3.2.5 Direct User Participation
 - 3.2.6 Multi-Agent Reinforcement Learning (MARL)
 - 3.2.7 Custom Agent Implementation
 - 3.3 Performance requirements
 - 3.4 Design constraints

Section 1.0 Introduction

Definition—This section of the SRS provides a broad overview of the proposed project, including the purpose, scope, and intended audience.

Section 1.1 Purpose

Definition—This section outlines the aim of the SRS and specifies the intended audience.

The purpose of this Software Requirements Specification (SRS) is to define the functional and non-functional requirements for the proposed Robot Sumo Simulation system. Which as outlined in the project plan is a comprehensive and accessible platform that will allow users to run and visualize simulated Robot Sumo matches, as well as train/evaluate reinforcement learning (RL) robotic agents [7]. This requirements document will specify what the system must do, its constraints, and the interfaces it must support; in sufficient enough detail to guide the system’s design, implementation, and testing.

Unlike a commercial software product, the Robot Sumo Simulation is intended as an academic and research contribution: a platform for exploring reinforcement learning, multi-agent systems, and Robot-Sumo-centric robotics simulation. In line with this, the intended audience for this SRS extends beyond the client and the development team to include the broader academic community, mainly students, researchers, or professors interested in RL-based robotics environments. Also included within this audience is the project development team (Adebare Kofo-Abayomi, Avlynn Wade, Faysal Zaki, and Joshua Oziel Fernández Alvarado) as well as the client and faculty advisor Dr. Hanks, who will evaluate the system’s functionality and its fulfilment of requirements through its development.

Section 1.2 Scope

Definition—this section defines the software product’s scope, including its functions, applications, goals, and benefits.

The software product being specified is the Robot Sumo Simulation, a Godot-based 3D physics simulation environment paired with an RL training and evaluation pipeline [5]. The system will allow users to configure Robot Sumo matches (robot design, sumo ring parameters, environmental effects) and run them both visually and in a “headless” mode for faster RL training. For RL training, users will create agents using established RL libraries: Gymnasium for the single-agent baseline, and PettingZoo for the extension into multi-agent reinforcement learning (MARL) once the baseline pipeline has been validated (Section 2.4) [3][4]. Users will also be able to participate directly in matches by controlling a robot themselves, to help in evaluation of an RL agent’s performance. Finally, if scope allows, users will be able to import custom agents or RL algorithms that conform to the system’s API (Section 2.4).

The system will not include physical robot hardware or a general-purpose game engine editor beyond what Godot provides. This outlined scope is consistent with the goals and objectives described in the development team’s Project Plan [7], which frames the system as an accessible platform for Robot Sumo simulation and RL experimentation.

Section 1.3 Definitions, acronyms, and abbreviations

Definition—This section provides the definitions for all terms, acronyms, and abbreviations required to interpret the SRS.

Term	Definition
RL	Reinforcement learning
MARL	Multi-agent reinforcement learning
Client	Dr. Hanks, faculty member at Florida Institute of Technology
Godot	The open-source game engine selected to build the Robot Sumo simulation environment.
Gymnasium	A Python library providing a standard API for single-agent reinforcement learning environments; used for the system's single-agent baseline.
PettingZoo	A Python library providing a standard API for multi-agent reinforcement learning; used for the system's MARL implementation.
RL APIs	General all-encompassing term for the currently utilized RL Training environment and library.
Agent	A software-controlled entity that observes the state of the simulation and selects/returns an action.
Headless Mode	A simulation run that does not include visual rendering, used to reduce computational overhead during RL training,
AI Panther	Florida Institute of Technology's supercomputing resource, proposed for use in the computationally expensive RL training.
Match/ Round	A single Robot Sumo competition in which two (or more robots) compete against each other.
Ring	The designated circular arena in which a Robot Sumo match takes place.
Slurm Job/ Script	A shell script containing the instructions and resource requirements used by the Slurm workload manager to submit, schedule, and execute a computing job on an HPC cluster (AI Panther within the context of this SRS document).

Section 1.4 References

- [1] "AI Panther Basics Tutorial," Florida Institute of Technology - Knowledge Base. Accessed: Sep. 24, 2026. [Online]. Available: <https://help.fit.edu/TDClient/39/Portal/KB/Article/21232/AI-Panther-Basics-Tutorial>
- [2] "What is the AI.Panther cluster?," Florida Institute of Technology - Knowledge Base. Accessed: Sep. 24, 2026. [Online]. Available: <https://help.fit.edu/TDClient/39/Portal/KB/Article/2831/What-is-the-AI-Panther-cluster>

- [3] “Gymnasium Documentation,” Gymnasium. Accessed: Sep. 24, 2026. [Online]. Available: <https://gymnasium.farama.org/index.html>
- [4] “PettingZoo Documentation,” Farama Foundation. Accessed: Sep. 24, 2026. [Online]. Available: <https://pettingzoo.farama.org/>
- [5] Godot Engine, “Godot Engine - Free and Open Source 2D and 3D Game Engine.” Accessed: Sep. 24, 2026. [Online]. Available: <https://godotengine.org/>
- [6] A. Kofo-Abayomi, A. Wade, F. Zaki, and J. O. Fernández Alvarado, “Robot Sumo Simulation: Design Document,” Dept. Elect. Eng. and Comput. Sci., Florida Institute of Technology, Melbourne, FL, USA, 2026.
- [7] A. Kofo-Abayomi, A. Wade, F. Zaki, and J. O. Fernández Alvarado, “Robot Sumo Simulation: Build AI Agents that Battle, Adapt, and Work as a Team,” Dept. Elect. Eng. and Comput. Sci., Florida Institute of Technology, Melbourne, FL, USA, 2026.
- [8] IEEE Recommended Practice for Software Requirements Specifications, IEEE Std 830-1998, 1998.
- [9] GeeksforGeeks, “Top 7 Python Libraries For Reinforcement Learning,” GeeksforGeeks. [Online]. Available: <https://www.geeksforgeeks.org/machine-learning/top-7-python-libraries-for-reinforcement-learning/>. [Accessed: Sep. 25, 2026].

Section 1.5 Overview

Definition—This section describes what the rest of the SRS contains and how it’s organized

- ***Overall Description (Section 2)***
Provides the high-level background on the system, including its context, major functions, intended users, constraints, and assumptions. However, it does not define any specific requirements.
- ***Specific Requirements (Section 3)***
Defines the system’s specific functional and nonfunctional requirements, including (but not limited to) interfaces, simulation and RL training requirements, performance requirements, and design constraints.

Section 2.0 Overall Description

Definition—This section describes the general factors that affect the product and its requirements, as well as necessary background information.

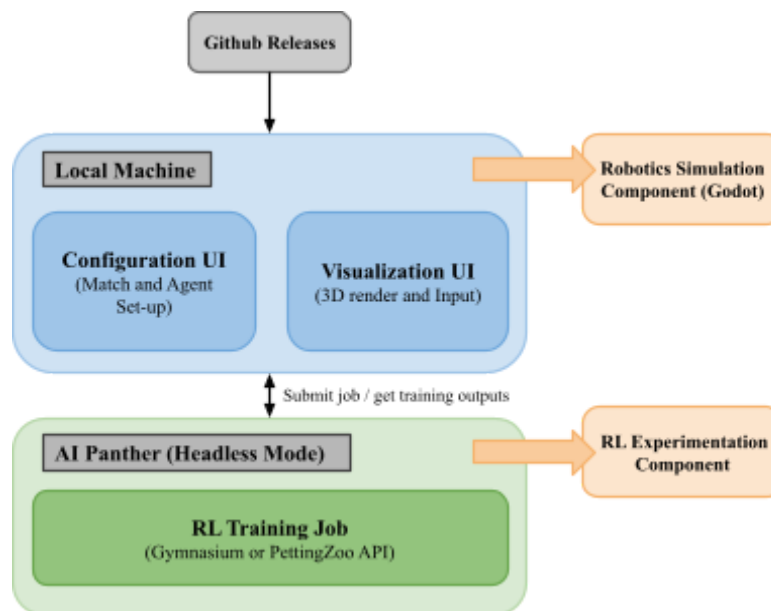
Section 2.1 Product Perspective

Definition—This section puts the software product into perspective with other related products.

The Robot Sumo Simulation is an independent system meant as a convenience tool for early experimentation and prototyping related to RL research; it is not a component of a larger system. Moreover, it is not meant to replace or extend any pre-existing Florida Institute of Technology software. However, it is intended to utilize the AI Panther supercomputing resource as an external processing tool for training RL agents in headless mode [1]. To summarize, AI Panther is more of a tool, not a fully integrated system component.

For the current phase of development, the system will consist of two primary components. The first is a Godot-based simulation environment modeling the physics of the Robot Sumo ring, the match environment, the robots themselves, and their interactions. The simulation will use a Gymnasium-compatible step-function interface to advance the environment, while Godot's main loop handles the underlying physics and input processing [3][5]. The second is the RL training layer that will be built using the existing Gymnasium API for baseline single-agent training. As of this semester, the training will occur separately on AI Panther, using the same Gymnasium-compatible step() interface, regardless of whether the simulation is running visually or in headless mode. The resulting trained parameters will be saved and loaded into the visualized simulation for observation/analysis.

Figure 1. Block Diagram for Robot Sumo Simulation Project



If AI Panther integration proves infeasible for the current timeline, training will be conducted locally on available developer hardware (using headless configuration). Additional goals include support for MARL via the PettingZoo API and custom RL agents through

user-defined parameters and weights; however, these features will be treated as optional goals rather than concrete requirements (Section 2.4).

Section 2.1.1 System Interfaces

Definition—This section identifies each system interface, its functionality, and how it supports the overall system requirements.

The Robot Sumo Simulation’s primary system interface is its connection to the AI Panther supercomputing resource. AI Panther is accessed via SSH connection to its login node (ai-panther.fit.edu), and job execution is managed by Slurm, with training jobs submitted from the login node as shell scripts specifying partition, node count, memory, and GPU allocation [1]. The job is queued and executed on an available compute or GPU node once resources are freed. Godot’s main loop will be structured so that simulation code can run either locally (for visualization purposes) or on AI Panther (headless mode), with no rendering dependencies in the training pipeline. Communication with AI Panther is limited to job submission and monitoring (sbatch and squeue), and retrieval of outputs: files containing model parameters and training results [1]. As mentioned previously, the saved data will be used as input for the visualization to display agent behaviors after training. Extending this system interface to support multi-agent training jobs as well as user-submitted training parameters will be discussed more thoroughly in Section 2.4.

Section 2.1.2 User Interfaces

Definition—This section defines the software’s user interface characteristics and usability requirements (layout, functionality, appearance, and ease of use).

The Robot Sumo Simulation provides two primary user interface modes within a single Godot desktop application (most likely distributed through GitHub Releases). The first is the configuration GUI: menus allowing users to select the simulation parameters, such as modular robot designs, ring dimensions and properties, and any environmental effects. The GUI will also allow the user to select their match type and which pre-trained AI agent(s) they wish to use. The second user interface mode is the visualization interface: a real-time, fully rendered 3D view of the Robot Sumo ring, in which the user observes the competing robotic agents. The visualization interface will also accept live keyboard and/or mouse input during a match, allowing a user to directly control a robot’s movements.

Menus and configuration screens should follow standard, consistent layout conventions (clearly labeled buttons, logically grouped settings, etc.) so that a user unfamiliar with the system can launch a simulation easily, and without consulting any outside documentation. In addition, any messages, such as winning/losing conditions or error messages, should be displayed clearly within the interface rather than requiring a user to inspect console outputs.

Section 2.1.3 Hardware Interfaces

Definition—This section identifies the hardware requirements needed for the software to operate.

The Robot Sumo Simulation does not require specialized or custom hardware; it is designed to run on standard desktop/laptop hardware capable of running a Godot-exported application with 3D rendering. User input is limited to standard keyboard and mouse operations for direct control of a robot within a rendered sumo match. Finally, RL training will be offloaded to AI Panther; however, in the fallback case, training may be conducted locally using standard hardware, with training time scaled accordingly.

Section 2.1.4 Software Interfaces

Definition—This section identifies the required software products/external software interfaces, including their purpose, specifications, versions, and sources.

Interface	[Specification, Version, Source]	Description
Godot Engine	[N/A, 4.7.2, Godot Engine]	The open-source game engine used to build the simulation environment and both user interface modes (Section 2.1.2). Godot is the foundational software platform for the application, with the simulation environment exposed to the RL training layer through the API-compatible step function logic established previously.
Gymnasium	[Gymnasium API, 1.3.0, Farama Foundation]	Defines the baseline single-agent RL interface (step function, observations, actions, etc.) which governs how an RL agent interacts with the simulation environment.
PettingZoo	[PettingZoo API, 1.27.0, Farama Foundation]	Defines the MARL interface. Considered a reach goal, but if implemented, would extend the existing RL interface outlined above.
Python	[Python Language Reference, 3.1.4, Python Software Foundation]	The programming language required to run the Gymnasium/PettingZoo APIs and the RL training programs submitted to AI Panther (internally by the development team). The Godot-Python bridge will be fully defined within the Design Document.

Section 2.1.5 Communication Interfaces

Definition—This section specifies the various interfaces for communication, such as local network protocols, etc.

The primary communication interface for the system is between the Godot application and AI Panther (Section 2.1.1). AI Panther uses SSH over TCP (network protocol) for discrete job submission and monitoring through Slurm, with outputs retrieved after the job is completed using standard file transfer techniques (scp) [1]. No persistent connection or additional network protocols are required, since the simulation interfaces operate locally.

Section 2.1.6 Memory

Definition—This section specifies any limits/characteristics on primary and secondary memory.

Memory requirements for the system can be broken into two sections. On the local Godot simulation side, the primary memory cost is the rendering of 3D visuals. However, this requires no specialized memory allocation beyond typical Godot rendering and should run on most commercial hardware. Specifically, the simulation will likely not render more than a dozen static low-polygon objects, and an average of two colliding bodies will be moving at one time.

On the AI Panther side of things, training jobs run as Slurm jobs with memory explicitly requested via the “--mem” directive, following the job’s workload requirements [1]. Based on published AI Panther documentation, the H200 SXM GPU nodes provide up to 12,288 GB of RAM, providing substantial memory capacity for large-scale RL training [2]. Output data is written to AI Panther’s shared or project storage and can be transferred to local storage after job completion. Since model parameter files and training logs will be relatively small in size/space, local storage requirements should be modest.

Section 2.1.7 Operations

Definition—This section should specify the normal and special operations required by the user

The system supports two modes of operation; the first is the interactive or user-initiated operation, which runs on the user’s local machine. Here, the user can launch the application, configure a match through the GUI interface, and observe or participate in that match through the visualization interface (Section 2.1.2). This mode requires user input throughout, with no unattended operations. The second operation mode is the unattended operation on AI Panther. As mentioned previously, a training job is submitted as a Slurm script and runs until it reaches completion, encounters an error, or reaches its allotted time (depending on the requested partition) [1]. This operation mode requires no additional user interaction beyond retrieving any outputs from AI Panther’s storage (Sections 2.1.5 and 2.1.6).

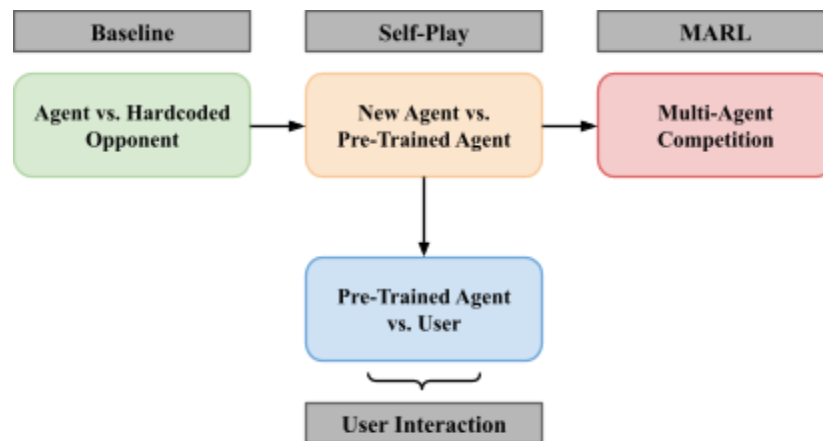
Section 2.2 Product Functions

Definition—this section provides a high-level summary of the major functions that the software will perform.

Function	Brief Description
Match Configuration	The user can configure a Robot Sumo match through many different parameters such as robot designs, ring settings, environmental effects, and the type of match.
Headless Mode for RL Training	The system supports a headless training mode where a simulation can run without rendering visual output (reduced computational overhead). This mode will be supported by the AI Panther supercomputing resource.

Progressive Model Training	The robotic agents are trained in stages of increasing difficulty. For example, starting with a hardcoded opponent and progressing to matches against previously trained agents. This will lay the foundation for future multi-agent training (See Figure 2.)
Agent Visualization	The user can observe a pre-trained agent within a 3D Robot Sumo simulation.
User Interaction	The user can choose to control a robot within a visualized match, which will help to evaluate a trained agent's performance.
Multi-Agent Reinforcement Learning	Reach Goal. The system will support multi-agent learning through the PettingZoo API (Section 2.4).
Custom Agent Importing	Reach Goal. The user can define and submit their own RL parameters and weights to use within the existing training pipeline (Section 2.4).

Figure 2. Progressive Agent Training Diagram



Section 2.3 User Characteristics

Definition—This section should describe those general characteristics of the intended users of the product (education, experience, technical abilities, etc.)

As mentioned briefly in Section 1.1, the Robot Sumo Simulation is intended primarily for an academic audience, including students, researchers, and instructors. To describe these users more precisely, this section adopts the same four user types defined in the project's design document: Developers, Trainers, Visualizers, and Players [6].

Developers	For the current phase of development, limited to the project team and the client/faculty advisor. Developers are expected to have competency with all system interfaces, Godot, Python, GitHub, AI Panther, and the Gymnasium/PettingZoo APIs.
-------------------	--

Trainers	Execute and delegate RL training jobs; are expected to understand basic RL concepts (agents, training, rewards/feedback, etc.), and have basic command-line familiarity (SSH and job submission). Trainers are encouraged to utilize a supercomputing resource for RL training; for the development team, this resource will be AI Panther. However, Trainers may utilize an alternative supercomputing resource provided that it's correctly integrated, as detailed in the Design Document [6].
Visualizers	Load and observe pre-trained agents or configure and run a Sumo match; require no prior experience with Godot, Gymnasium, or PettingZoo, and no command-line experience.
Players	Participate directly in a match by controlling a robot, expected to have familiarity with the Robot Sumo rules and basic game controls (keyboard/mouse inputs).

Section 2.4 Constraints

Definition—This section provides a general description of any other items that will limit the developer's options.

The following features are considered reach/optional goals for the current phase of development and are not committed requirements within this SRS: multi-agent reinforcement learning support via PettingZoo, and user-defined custom RL agents. Implementation depends on the remaining project scope after the baseline single-agent pipeline is validated. Furthermore, for user-defined agents, the development team would need to resolve any integration issues with the planned AI Panther training pipeline.

The system is further constrained as a locally-run application, rather than a web-based application. The RL training component is also constrained by AI Panther's policies: jobs are subject to specific time limits (45 minutes to unlimited time), and shared storage is purged after a certain time, which will require timely output file transfer. Finally, since AI Panther access is an institutional resource tied specifically to Florida Institute of Technology credentials, the system is constrained so that end users are not assumed to have personal AI Panther access. However, the system is not strictly dependent on AI Panther, and alternative supercomputing resources may be used for RL training, as described in the Design Document [6].

Section 2.5 Assumptions and Dependencies

Definition—This section should list factors that affect the requirements stated in the SRS.

A. AI Panther Access: The development team is assumed to retain access to AI Panther throughout the project's development (August 2026 - May 2027). This dependency applies only to the training pipeline, not end users.

B. Software Compatibility: Godot, Gymnasium, PettingZoo, and supported Python environments are assumed to remain compatible throughout development.

C. Training Data Storage: AI Panther shared storage is subject to automatic purging; the development team is responsible for retrieving and organizing training outputs.

D. User Hardware: End users are assumed to have access to standard commercial hardware capable of rendering a 3D Godot-exported application.

Section 3.0 Specific Requirements

Section 3.1 External interface requirements

Definition—This section defines the specific external interfaces the system must support, including user, hardware, software, and communications

3.1.1 User interfaces (Configuration GUI and Visualization)

1. The system shall provide a configuration GUI interface that allows users to set robot design options, ring dimensions, and environmental effects before running a match.
2. The configuration GUI interface shall allow the user to select a match type: agent vs. hardcoded opponent, agent vs. agent, or user-controlled robot vs. agent.
3. The configuration GUI interface shall allow the user to select a pre-trained agent to load in for visualization.
4. The system shall provide a visualization interface which displays a rendered 3D view of the Robot Sumo ring during a match.
5. The system's visualization interface shall accept keyboard and/or mouse input during a match to allow a user to control a robot themselves.
6. The system shall display any messages (error, winning, etc.) clearly and directly within the interface; the user should not have to inspect logs or console output.
7. The system shall visually indicate the outcome of a match.
8. The configuration GUI shall provide controls for starting, pausing, or ending a match.
9. The visualization interface shall display relevant agent information during a match, such as the active agent and selected model.
10. A user unfamiliar with the system shall be able to navigate through the functionalities without significant issue.

3.1.2 Hardware interfaces

11. The system shall accept keyboard and mouse input as the only required hardware components.
12. The local visualization application shall be able to run on standard consumer/developer hardware (laptop or PC).

3.1.3 Software interfaces

13. The system shall be built using Godot (version 4.7.2), obtained from the official Godot Engine distribution site.
14. The system's single-agent baseline training shall conform to the Gymnasium API (version 1.3.0) for defining the step function, observations, feedback, and reward systems.
15. The system's RL training algorithms shall be implemented using a well-documented RL library [Stable-Baselines3 (SB3), PyTorch, RLlib, etc].
16. The Godot simulation shall communicate with the Python-based RL training pipeline via a bridging tool [Godot to Python, etc.]

3.1.4 Communications interfaces

17. The system shall submit RL training jobs to AI Panther via SSH access to its login node, and Slurm job submission.
18. The system shall retrieve completed training job output files (model parameters or training logs) from AI Panther via standard file transfer (scp).
19. The system shall require no persistent or real-time network connection during local visualization or configuration.

Section 3.2 Functional requirements

Definition—This section defines the specific functional requirements of the system

Section 3.2.1 Match Configuration

20. The system shall allow the user to configure modular robot design parameters before running a match.
21. The system shall allow the user to configure ring dimensions and properties before running a match.
22. The system shall allow the user to configure environment settings before running a match.
23. The system shall verify match configuration inputs and notify the user of any invalid actions before running a match.

Section 3.2.2 Single-Agent RL Training

24. The system shall support running the simulation in headless mode (non-rendered) for RL training.
25. The system shall train a single agent using the Gymnasium API against an opponent.
26. The system shall save trained model parameters as output upon completion of a training job.
27. The system shall support running training jobs on AI Panther via Slurm job submission.

- 28. The system shall support running training locally on developer hardware if AI Panther access is unavailable.
- 29. The system shall train an RL agent using a well-documented RL algorithm (single-agent approach, CTDE, the evolutionary approach, etc.)

Section 3.2.3 Progressive Training

- 30. The system shall support training an agent against a static, hardcoded opponent.
- 31. The system shall support self-training by allowing an agent to compete against a previously trained version of itself (self-play).
- 32. The system shall use consistent random seeding during training to support reproducibility.

Section 3.2.4 Pre-Trained Agent Visualization

- 33. The system shall allow the user to load a saved, pre-trained agent's parameters into the visualization interface.
- 34. The system shall render the ring and its agents correctly, matching any configuration when running a 3D match visualization.
- 35. The system shall notify the user if a selected pre-trained agent file fails to upload.

Section 3.2.5 Direct User Participation

- 36. The system shall allow the user to control a robot's movement and actions in real time via keyboard/mouse input during a visualized match.
- 37. The system shall allow a user-controlled robot to compete against a trained agent or a hardcoded opponent.
- 38. The system shall determine and display match outcomes (win or loss conditions) regardless of whether or not a match involves a user-controlled robot.

Section 3.2.6 Multi-Agent Reinforcement Learning (MARL)

- 39. (Reach) The system shall support training multiple simultaneous agents using the PettingZoo API (version 1.27.0).

Section 3.2.7: Custom Agent Implementation

- 40. (Reach) The system shall allow a user to define and export a configuration file or custom training parameters and weights.
- 41. (Reach) The system shall allow a user-defined configuration file to be injected into the existing training pipeline without requiring execution of user-submitted code on AI Panther.

Section 3.3 Performance requirements

- 42. The visualization should render at a minimum of 30 frames per second during a match.

- 43. A single-agent training job shall request no more memory than AI Panther's partition-specific time limits.
- 44. The local application shall launch and be ready for user interaction within 10 seconds of being started.
- 45. The configuration GUI interface shall reflect a user's input changes (parameter adjustments) with minimal delay.
- 46. Loading a pre-trained agent into the visualization interface shall complete within 30 seconds to a minute.

Section 3.4 Design constraints

- 47. The system shall be distributed as a locally-run application rather than a web-hosted application.
- 48. The system shall not require end-users to have access to AI Panther.
- 49. Support for MARL and custom RL agents shall remain outside of the committed scope for the current development phase and will be implemented only if time allows.